



Figure 1: Push-buttons (or just 'buttons')



Figure 2: CheckBox

```
#endif // DIALOG_H
```

This file is *dialog.cpp*:

```
#include "dialog.h"
#include "ui_dialog.h"
#include <QtCore>

Dialog::Dialog(QWidget *parent):
    QDialog(parent),
    ui(new Ui::Dialog)
{
    ui->setupUi(this);
    connect(ui->clickme, SIGNAL(clicked()), this, SLOT(button_clicked()));
    connect(ui->clickme, SIGNAL(pressed()), this, SLOT(button_pressed()));
    connect(ui->clickme, SIGNAL(released()), this, SLOT(button_released()));
}

Dialog::~Dialog()
{
    delete ui;
}

void Dialog::button_clicked()
{
    qDebug() << "Button Clicked";
}

void Dialog::button_pressed()
```

```
{
    qDebug() << "Button Pressed";
}

void Dialog::button_released()
{
    qDebug() << "Button Released";
}
```

This file is *main.cpp*:

```
#include <QtGui/QApplication>
#include "dialog.h"
int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    Dialog w;
    w.show();
    return a.exec();
}
```

In *dialog.h*, we declared three private slots—*button_clicked()*, *button_pressed()* and *button_released()*. Their definition in *dialog.cpp* simply prints a debug message stating what event occurred. In the *Dialog* class constructor, we connected these slots with the *clicked()*, *pressed()* and *released()* signals from the *clickme(QPushButton)* object. The pointer “*ui*” refers to the dialogue window; using it, we can access its child widgets—just as we did to access the *clickme* push button in the *connect()* lines.

The first argument of the *connect()* function (a static function from the class *QObject*) is the source object of the signal; the second is the signal itself; the third is the signal-receiving object, and the last is the slot function. Compile and execute the above code. When the dialogue window comes up, click the *clickme* button and check that you find the appropriate debug output on the console. The output shows that three signals are emitted when a *pushbutton* is clicked: first *pressed*, then *released* and finally *clicked*.

Message-boxes

Before moving to other GUI classes, we need to cover one important class, *QMessageBox*, which is used very often. For example, when you try to close an unsaved document, the pop-up prompt asking if you want to save/close the document (in Qt) uses this class. Its main use is as an alert; some text and even icons can be displayed on it, and a user might even be asked to enter some text through it. *QMessageBox* also supports a message severity level, which displays different icons along with the text message. Here is an example:

```
int res = QMessageBox::information(this, "Option", "Want to choose
" + item + "?", QMessageBox::Yes | QMessageBox::No );
switch(res)
{
    }
```